

Basic Concept of Overall Framework (3rd draft)

Mizuho-DL Financial Technology Co., Ltd.
September 4, 2006

The copyright for this material belongs to Mizuho Securities Co., Ltd. and unauthorized citation or reproduction, in whole or in part, for any purpose is prohibited. It is not to be disclosed to third parties without the prior written consent of Mizuho Securities Co., Ltd. This material was prepared based on information our company qualified as reliable and determined to be accurate, however, no guarantee is given as to its accuracy or completeness. As your company will use this material at your own discretion and under your own responsibility, when necessary, it should be handled following consultation with your legal, accounting and tax advisors.

Note: Changes subsequent to the preparation of this document (since September 4, 2006) have not been reflected. Consequently, the information, contained in the basic designs and other such documents and in the source code, may not match with the information in this manual.

1.	Introduction.....	4
2.	M-Library summary.....	4
2.1.	Composition.....	4
2.2.	Summary of the pricing by the M-Library	4
2.3.	Entity and attribute classes.....	5
2.4.	Calculation classes.....	6
2.5.	Reference between entities.....	7
2.6.	Change notification function.....	7
3.	Class naming rules.....	8
4.	Overall image.....	8
5.	Transaction expression.....	9
5.1.	Transaction.....	9
5.2.	Leg.....	9
5.3.	Trigger provisions.....	9
5.4.	Callable provisions.....	9
5.5.	Cash flow information	9
5.6.	Creating cash flow information.....	10
5.7.	Portfolio.....	10
6.	Path expression and creation	12
6.1.	MC path expression	12
6.2.	Expression of path elements and curve classes.....	13
6.3.	Model.....	13
6.4.	Input.....	13
7.	Pricing (MC).....	15
7.1.	Index expression	15
7.2.	Payoff calculation	15
7.3.	PV calculation	15
7.4.	Parallelization processing	16
8.	Tree expression and creation	18
8.1.	Tree expression	18
8.2.	Tree generation.....	18
9.	Pricing (tree).....	18
9.1.	Index calculation.....	18
9.2.	Payoff calculation	18
9.3.	PV calculation.....	18

10. Pricing (analytical solution).....	20
11. Risk index computation	20
11.1. Computation of the portfolio pricing and risk index.....	20

1. Introduction

This manual provides a summary of the framework for the interest rate evaluation by the M-Library.

2. M-Library summary

This chapter provides a summary of the M-Library to help you to understand the next and subsequent chapters.

2.1. Composition

The M-Library is composed of the respective sub-libraries ME, MB, MF, MD, MM and MV.

The respective sub-libraries are summarized here following.

ME: Provides a function that controls exceptional processing.

MB: Provides a function that maintains the relationship between data expressions and data.

MF: Provides a function for expressing financing related information. For example, the calendar and yield curve classes are included.

MD: Provides a function that acquires data from external data sources.

MM: Provides a function for numeric calculation. The functions of interpolation, matrix operations, numerical integration and random numbers are assigned to it.

MV: Provides various model and evaluation related functions.

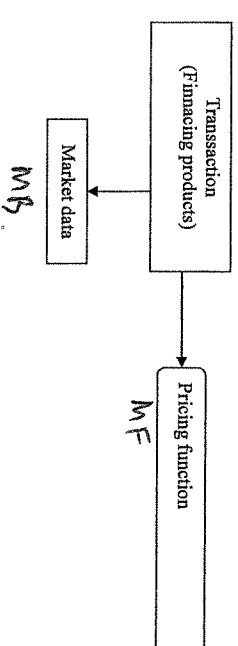
MA: Application / Set-up.

2.2. Summary of the pricing by the M-Library

The below figure shows the relationship with the respective objects in the pricing using the M-Library.

The information determining the transaction details (coupon, principal, etc., hereinafter called the attribute data) is held in the transaction object. The transaction object references the market data object and the pricing function object required during the evaluation.

The pricing function object acquires and prices the required attribute data from the transaction object and market data.



Though there are sometimes more complex structures, depending on the type of transaction, they are essentially composed of the relationships shown in the above figure.

Here following is a somewhat more detailed description of the respective objects shown in the above figure.

2.3. Entity and attribute classes

The transaction (product) and market data are expressed by the MBEntity class and other derivative classes (generically referred to as entity classes). Though the types of attribute data necessary for the respective transaction classifications differ, in principle, the M-Library does not create a derivative class for each type of transaction, but instead creates derivative classes by large categories (transaction, yield curve, volatility, credit, path, etc.).

For example, the knock-out option requires attribute data related to the knock-out, in addition to the attributes required for the plain option, however, it does not derive the plain option and then create the knock-out option, but rather expresses both in the same class.

The below table contains examples of MBEntity derivative classes.

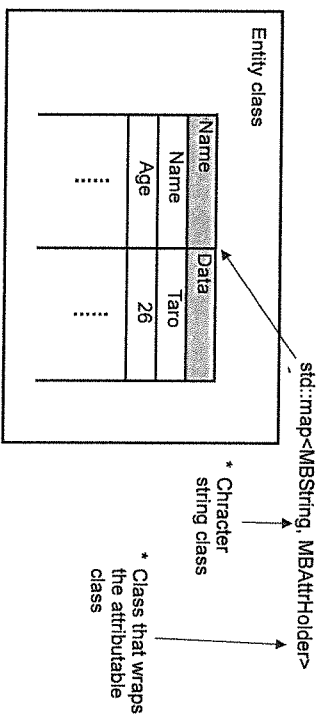
Class	Explanation
MBValuableEntity	Expresses classes that express transactions (financial products), in other words, transactions subject to pricing.
MBYieldCurve	Classes that express yield curves
MBSingleCredit	Classes that express the credit of individual issues

The derivative class of the MBAAttribute class (generically referred to as the attribute class) expresses the attribute data of the transaction (product) and the market data. The below table contains examples of MBAAttribute derivative classes. These are used to express attribute data for coupons, principals and maturities.

Class	Explanation
MBAttrBool	Class that expresses the Boolean(bool type) data
MBAttrInt	Class that expresses the integer (int type) data
MBAttrDouble	Class that expresses double precision floating point (double type) data
MBAttrString	Class that expresses character string data
MBAttrDate	Class that expresses date data
MBAttrDateTime	Class that expresses date and time data

Apart from basic data types such as the above, the attribute class also includes data indicating calendars, random number generators and functions, and data indicating references to the later mentioned evaluation classes and references to other entities.

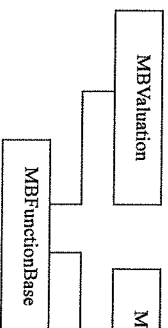
The entity class uses the map class of the C++ STL (standard template library) to hold an optional aggregation of attribute classes in the <name (key) and data (value)> format. Various transactions (financing products) can be expressed in one type of class by adopting this sort of mechanism.



Bool, int, double, ...

2.4. Calculation classes

The MBFunctionBase class is defined as the basal class of the classes that perform some calculation (also including pricing and calibration). The MBValuation class is defined as the basal class of the classes that price some transactions (financing product). In addition, the MBProcedure class is defined as the basal class of the classes that perform such functions as CF generation, and calibration. The MBValuation and MBProcedure classes succeed the MBFunctionBase.



Cashflow generator
YC, ...
derive this

These classes (see for reference) also define the below MBAAttribute derivative classes, so that they can be handled as attribute data of the entity classes.

MBAttrValuation:

The attribute class that indicates the reference to the pricing function class

MBAttrProcedure:

Attribute class that indicates the reference to the procedure class (class that performs CF creation, interest rate path creation, etc.)

Apart from the above as well, the random number class, function class ($R^R \rightarrow R$), interpolation class, optimization class, numerical integration, etc. have been prepared as MBFunctionBase class derivative classes.

2.5. Reference between entities

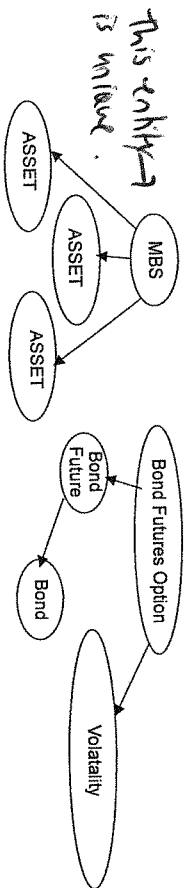
In order to express a transaction (financing product), it must indicate the classes where a certain entity references other entities. The attribute class includes the below derivative classes indicating the references to the other entity classes.

MBAttrReference:

Indicates reference to the entity class

MBAttrMultiReference:

Indicates reference to multiple entity class



2.6. Change notification function

When the referenced entity has changed (when attribute data has been subject to some change), this function notifies the source reference entity.

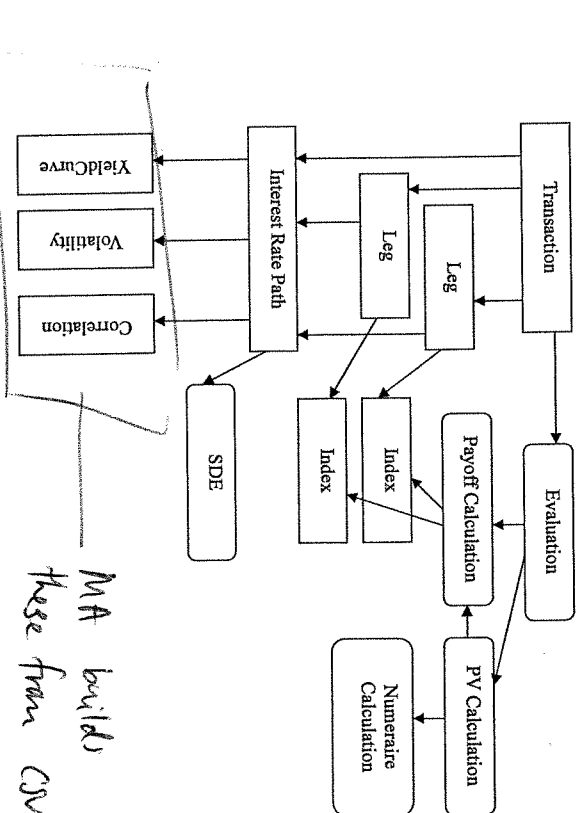
The existence of this function makes it possible to avoid useless re-calculation. For example, when calculating the foreign exchange delta of several transactions, though only the transaction affected due to the shifted foreign exchange rate must be re-priced, it is determined on the application side for each individual transaction whether the concerned transaction is dependent on foreign exchange, which is troublesome. The MBAttrProcedure value method is designed, so that it is only re-calculated when a change notification has been received.

3. Class naming rules

In principle "ir" will now be added following the two characters indicating the sub-class (with some exceptions) for class names to be newly added. For example, MFrO◇△, etc.

4. Overall image

This shows a relational diagram of the respective objects in the interest rate derivative pricing by the MC model.



Symbols:

Entity class (MBEntity class and its derivative class)

Calculation class (derivative class of the MBFunctionBase class)

5. Transaction expression

In this chapter we will discuss the methods of expressing transactions and the information related to them.

5.1. Transaction

Transactions are expressed by the `MfValuableEntity` class. Hereinafter it will be called the transaction entity.

The transaction entity has the input information required for pricing, for example the information indicated below, as attribute data.

Evaluation related provisions (evaluation date)

(Reference to) Leg

(Reference to) Callable provisions

(Reference to) Trigger provisions

Moreover, the evaluation results, such as the PV of the concerned transaction and the risk index are stored as attribute data of the transaction entity.

5.2. Leg

Transactions have one or multiple legs. Legs are expressed by the `MBEntity` class. Hereinafter it will be called the Leg Entity.

The Leg Entity has the following information as attribute data.

Cash flow information (see 5.5)

(Reference to) Index entity (see 7.1 for reference)

In addition, the evaluation results of the concerned leg (PV and risk index) are stored as attribute data of the Leg Entity.

5.3. Trigger provisions

Triggers are expressed by the `MBEntity` class.

5.4. Callable provisions

Callable provisions are expressed by the `MBEntity` class.

5.5. Cash flow information

Cash flow information is the information required for cash flow calculation. Cash flow information is composed of multiple cashlets.

Cashlets are expressed by the `MBEntity` class. The `MBAttrMultiReference` class has been provided in the `M-Library` to hold multiple `MBEntity` class references as attribute data.

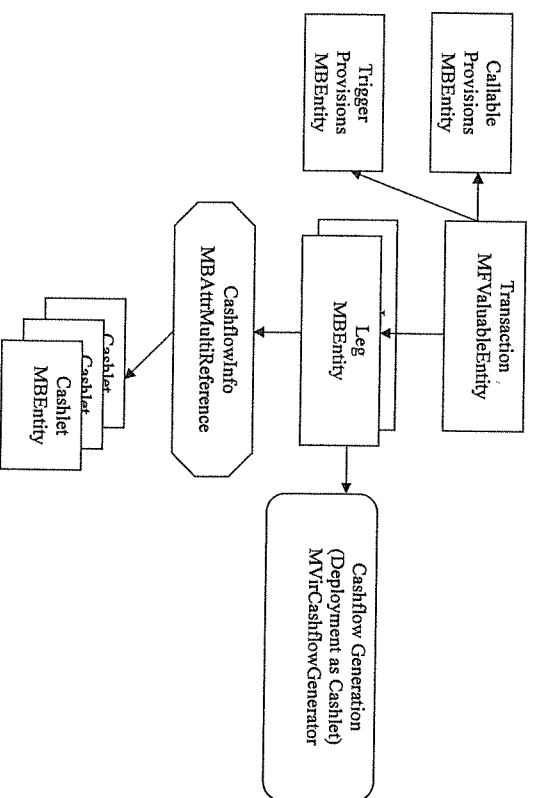
With the exception of special cases¹, cashlets do not hold the payoff values themselves, but instead hold the information for calculating the payoffs. Consequently, the expression method for the cash flow information does not depend on the pricing technique (MC or tree).

In addition, the `MBEntity` class uses the `std::map` class to store the attribute data (attribute class). At the time of the payoff calculation, a measure will be devised, so that the time required for this access does not become a bottleneck for the MC simulation.

5.6. Creating cash flow information

The `MVtrCashflowGenerator` class creates the above cash flow information in the typical pattern. `MVtrCashflowGenerator` class succeeds the `MBProcedure` class.

The relationship of the respective objects is illustrated below.



5.7. Portfolio

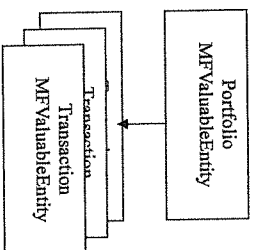
The consolidation of one or several transactions is called a portfolio. Portfolios are expressed by the `MfValuableEntity`. Hereinafter it will be called the portfolio entity.

The portfolio concept will be introduced, because in some cases consolidating several transactions for calculation is more efficient when calculating risk indices (see Chapter 11).

The portfolio entity has the following sort of information as attribute data.

¹ Case where the payoff is a fixed value

(Reference to) Transaction
Calculation index related provisions



6. Path expression and creation

6.1. MC path expression

In this manual, the MC path indicates the multiple paths to be used at the time of the MC. The MC path expresses through the MFiPathEntity class. Hereinafter it will be called the path entity. MFiPathEntity class succeeds the MBEEntity class.

In addition, this class can also express multiple types of MC paths (e.g.: yen interest rate path, dollar interest rate path and Fx path) in one object².

The attribute information held by the MFiPathEntity is as follows.

- (Reference to) Model # Several models can be held
- Path calculation conditions (tenor, term information, cache size, etc.)
- (Reference to) Volatility and correlation
- Random number generation module
- (Reference to) Path generation class

This class provides the below sorts of functions.

- Generates paths
It generates paths using the model class that has been set as the attribute information.
- Cache
It holds the generated path in memory up to a previously set cache size (specified by the number of paths).

For example, consider the case where it evaluates transaction A and transaction B in this order. In addition, it adopts 1 million as the number of MC simulations and 500 thousand paths as the cache size. Of the paths generated when transaction A is evaluated, the first 500 thousand are not erased, but rather are held in memory. In the evaluation of transaction B, the equivalent of the first 500 thousand is regenerated from the 500001st path, using the paths on the cache. Here, the evaluation class, provides the MFiPathEntity class with a mechanism permitting access in the same manner, without changing the processing regardless of whether or not there is a path on the cache.

- Random number control
It has a function that generates random numbers with a correlation.
It also has a function that holds the random number status. This function is required when using the previously described cache function to create from paths along the way.
- Negative correlation method
Provides a path generation function that uses negative correlation method. The cache holds the positive and negative paths separately.
- PCA
Offers the PCA function

² Because there is a correlation among the yen interest rate, the dollar interest rate and foreign exchange, several types of MC paths are handled by one object, in consideration of convenient random number control.

6.2. Expression of path elements and curve classes

Here, path element designates the value at a certain point in time of a single path. In other words, the `std::vector<path element*>` indicates a single path and `std::vector<std::vector<path element*>>` indicates MC paths.

In the case of `Fix` and `Equity`, the path element is a scalar value expressed by a floating point pattern, whereas, in the case of the interest rate, it is a curve with a term structure.

The `MFPATHElementBase` is created as the basal class of the classes indicating the path elements, in order to handle different types of paths in a uniform way.

The following two are provided as derivative classes of the `MFPATHElementBase` class.

The `MFiScalar` class expresses the floating point pattern.

The `MFiCurve` class is a base class of the class that expresses the curve at a certain point in time t . It creates derivative classes as required. This class has the function of returning the discount bond price $P(T-t)$.

6.3. Model

The `MMSDBase` class is a base class that expresses the SDE (stochastic differential equation) and succeeds the `MBFunctionBase` class.

The function of this class is as follows.

Setting of the initial value

Setting of the drift coefficient and the diffusion coefficient

(Micro) time steps

The derivative classes of these classes are added to the M-Library, as required.

The model and time step integration methods (e.g. Euler-Maruyama approximation) are implemented for the derivative classes of the `MMIDSDE` and `MMSDBase` classes.

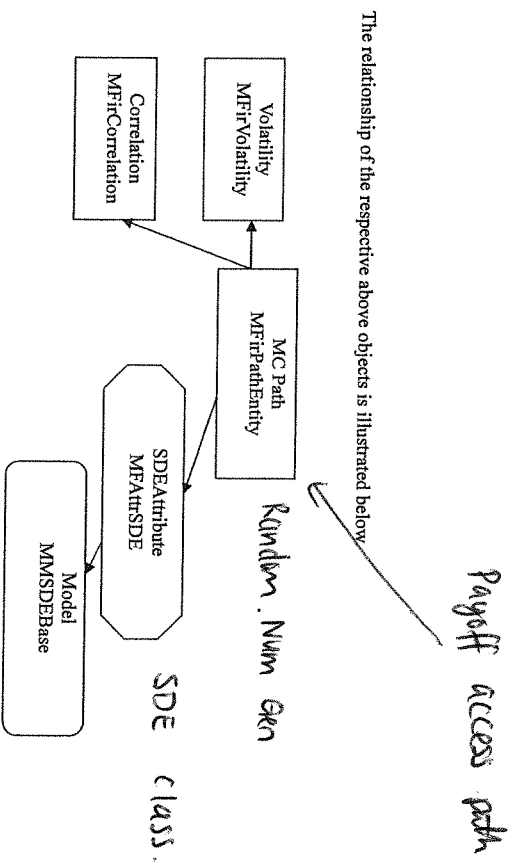
In addition, the `MFAITTSDE` class will be added to the M-Library, to enable it to handle the `MMSDBase` class and its derivative class as attribute data of the entity class.

6.4. Input

It adds the following classes that express the volatility and correlation to the M-Library. These succeed the `MBEntity` class.

`MFiVolatility`

`MFiCorrelation`



7. Pricing (MC)

7.1. Index expression

The `MVIRIndexEntity` class expresses the reference index for calculating the payoff and succeeds the `MBEntity` class. Hereinafter it will be called the index entity.

The attribute information held by the `MVIRIndexEntity` is as follows.

Conditions of the index to be computed
(Reference to) Path entity

This class offers the below sorts of functions.

- It creates index paths
 - Cache
- It has the same cache function as the `MVIRPathEntity`. It also compatible with the paths that use negative correlation method.

7.2. Payoff calculation

The `MVIRPayOff` class calculates the payoff and succeeds `MBFunctionBase` class. It creates derivative classes of this class, as required.

This class offers the below functions.

- Index setup
Sets the required attribute information, such as the reference date, for the index entity, based on the cash flow information.
- Payoff Calculation
Calculates the payoff, by computing the coupon from the index. It also decides on the trigger related processing and early exercise, as required³.

The coupons are generally calculated by multiple index operations, however, this class offers arithmetic functions, such as the four basic arithmetic operators and `Max()`/`Min()`.

When more complex operations are necessary, it responds individually by creating derivative classes.

7.3. PV calculation

The `MVIRCFValue` class calculates the PV of the payoff and succeeds `MBFunctionBase` class. This class calculates the numerate corresponding to the time of payoff generation and computes the PV.

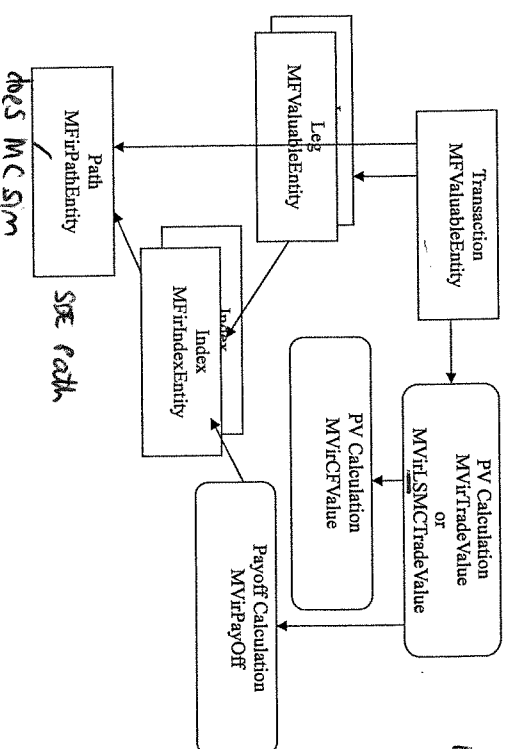
The `MVIRTradeValue` class calculates the PV of transactions using the MC method and succeeds the `MBValuation` class. This class is set in the transaction entity.

³ Assuming that the conditional expression for determining early exercise, such as the regression coefficient for the LSMC, is already known. The `MVIRLSMCTradeValue` class performs the LSMC, while performing regression analysis backwards.

This class holds the pointers of the `MVIRPayOff` and `MVIRCFValue` classes as member variables. This class is the origin of the pricing; it does the pricing using the `MVIRPayOff` and `MVIRCFValue` classes and stores the result in the leg entity and the transaction entity.

The `MVIRLSMCTradeValue` class calculates the PV of transactions using the LSMC and succeeds the `MBValuation` class. This class is set in the transaction entity. This class also uses the `MVIRPayOffCalculator` and the `MVIRCFValue` classes.

The relationship of the respective above objects is illustrated below.



7.4. Parallelization processing

MPI is used for parallelization.

The `MVIRMPIDataManageTool` class offers functions related to the sending and receiving of entity data between processes.

The `MVIRMPITradeValue` and `MVIRMPILSMCTradeValue` are the parallelization versions of respective `MVIRTradeValue` and `MVIRLSMCTradeValue` classes.

The `MVIRMPITradeValue` (`MVIRMPILSMCTradeValue`) holds the `MVIRTradeValue` (`MVIRLSMCTradeValue`) class as the member variable.

When the value method of the `MVIRMPITradeValue` class is called in the main process, this class sends entity information and the number of starts and the number of ends of the path for the MC method.

It transmits to the sub-processes.

Here, when transmission information has not been specified, it transmits the transaction entity information in which this class is set and all the entity information which is referred directly or indirectly to the sub-process.

In addition, when transmission information has been specified, it transmits only the specified information. For example, when the strike rate has been changed and it is to be recalculated, this function is effective for efficiently calculating it.

The pricing is done independently in the respective processes. Following pricing, the sub-process sends the results to the main process and they are tabulated in the main process.

When parallelization processing is performed for the LSMC, the main process evaluates it using the MVMPTLSMCTradeValue class and the MVMPTLSMCTradeValue is used by the sub-process. At this time, it sends the regression coefficient for the optimum exercise that was computed by the main process to the sub-process.

8. Tree expression and creation

8.1. Tree expression

The 2D3 item tree is expressed by the MVMPTLSMCTrimomialTree class.

This class maintains the below sorts of functions.

- Tree structure
- Status value in each node
- Transition probability
- Shift function
- AddaptiveMesh

8.2. Tree generation

The MVMPTLSMCTreeBase class is the base class of the class that assumes generation of the 2D3 item tree and succeeds the MBFunctionBase class.

It creates derivative classes of this class, as required. In this class, the method related to calculation of the transition probability is defined as a pure virtual function and is implemented in the derivative class.

9. Pricing (tree)

9.1. Index calculation

The MVMPTLSMCTree class computes the index on the specified node of the tree.

9.2. Payoff calculation

The MVMPTLSMCTree class computes the payoff on the respective node of the tree. This class uses the MVMPTLSMCTree class to calculate reference index to calculate the payoff, based on the cash flow information (see 5.5).

9.3. PV calculation

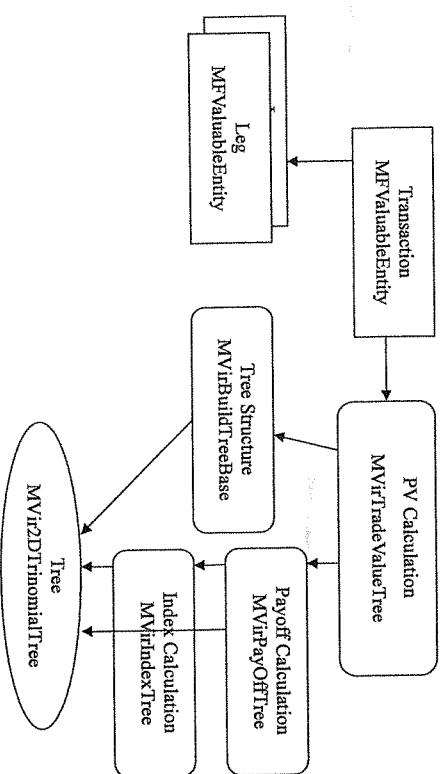
The MVMPTLSMCTree class does the pricing according to the tree. This class is set in the transaction entity.

When the value method of this class is called, it uses the MVMPTLSMCTreeBase class to generate the tree, then uses the MVMPTLSMCTree class to calculate the payoff and to calculate the PV.

In addition, it offers barrier processing and Richardson's extrapolation function for improving precision.

Experimental

The relationship of the respective above objects is illustrated below.



10. Pricing (analytical solution)

The derivative classes of the below MValuation classes are added to the M-Library.

MVCapFloorLMM class

Class that computes the analytical solution of the cap and the floor of the Libor market model.

MVSwapionLMM class

Class that computes the analytical solution of the Swapion by the Rebonato of the Libor market model.

MVSwapionSMM class

Class that computes the analytical solution of the Swapion of the swap market model.

MVCapFloorG2XX class

Class that computes the analytical solution of the cap and the floor of the G2++Model.

MVSwapionG2XX class

Class that computes the analytical solution of the Swapion of the G2++Model.

11. Risk index computation

11.1 Computation of the portfolio pricing and risk index

When calculating the risk index, processing by procedure 2 is more efficient than procedure 1 when using the path cache function.

Procedure 1

Price transaction 1 by the original with the original market data

Shift the market data, and then price transaction 1

Return to the original market data, and then price transaction 2 Shift the market data, and then price transaction 2

....

Procedure 2

Price transaction 1, transaction 2... with the original marketing data

Shift the market data, and then price transaction 1, transaction 2....

The MVirPortfolioValue class evaluates multiple transactions by procedure 2, and computes the PV and risk index. This class is set in the portfolio entity.

End